

Project 2: Backend Engineering

Project Title: Multi-Tenant SaaS Backend

Duration: 21 April 2026 – 19 May 2026

Deadline: 19 May 2026

1. Project Overview

Most modern software platforms (such as CRMs, project management tools, and HR systems) are built as SaaS (Software as a Service) products. These platforms serve multiple companies (called tenants) using a single system, while ensuring that each company's data remains completely separate and secure.

In this project, you will build a multi-tenant backend system using Django and APIs. Each company should be able to register, manage its users, and work with its own projects and tasks—without accessing any other company's data.

2. Problem Statement

Companies need a centralized platform where:

- Multiple organizations can register and use the same system
- Each organization's data is completely isolated and secure
- Teams within each company can collaborate using roles and permissions
- All activities are tracked for accountability

Your task is to design and build a backend system that fulfills these requirements in a scalable and secure way.

3. Project Objectives

By completing this project, you should be able to:

- Understand and implement multi-tenant architecture
- Build secure and scalable APIs
- Design relational databases for real-world applications
- Implement authentication and authorization systems
- Track user activity through logging mechanisms

4. What You Need to Build

You are required to build a **backend API system** that supports multiple companies (tenants). Each tenant should have:

- Its own users
- Its own projects
- Its own tasks

No data should overlap between tenants.

5. Functional Requirements

Your system must include the following features:

5.1 Company (Tenant) Management

- Users should be able to:
 - Register a company
 - Create an initial admin account
- Each company must have a unique identity
- All data must be linked to a specific company

5.2 User Roles and Permissions

Implement role-based access control within each company:

- Admin:
 - Full access to all data
 - Can manage users
- Manager:
 - Can create and manage projects and tasks
- Employee:
 - Limited access (only assigned tasks/projects)

Permissions must be enforced at the API level.

5.3 Authentication System

- Implement secure login/signup
- Use JWT (JSON Web Tokens) for authentication

- Ensure protected routes require authentication
- Tokens should expire and be refreshable (bonus)

5.4 Project and Task Management APIs

Create full CRUD (Create, Read, Update, Delete) APIs for:

Projects

- Create project
- Assign users
- Update project details
- Delete project

Tasks

- Create tasks under projects
- Assign tasks to users
- Update status (e.g., pending, in progress, completed)
- Delete tasks

5.5 Data Isolation (Critical Requirement)

This is the most important part of the project.

- A user from Company A must NEVER access data from Company B
- All queries must be filtered by tenant
- Enforce this at:
 - Database level (foreign keys, relations)
 - API level (query filtering)

5.6 Audit Logging System

Track all important actions in the system:

- User login/logout
- Project creation/updates
- Task changes

Each log should include:

- User
- Action

- Timestamp
- Related object (project/task)

6. Technical Requirements

You must use:

- Django
- Django REST Framework (for APIs)
- PostgreSQL (preferred for relational data)
- JWT-based authentication

7. Suggested System Design (Guidance)

You may structure your system as follows:

- Company (Tenant)
- User (linked to Company)
- Project (linked to Company)
- Task (linked to Project + User)
- AuditLog

Ensure proper relationships using foreign keys.

8. Advanced Requirements

To achieve a high evaluation score, you must implement:

8.1 API Rate Limiting

- Prevent abuse of APIs
- Limit number of requests per user/IP

8.2 Soft Delete Functionality

- Instead of permanently deleting records:
 - Mark them as deleted
- Allow recovery if needed

8.3 Background Jobs (Task Queue)

- Use background processing for:
 - Sending emails
 - Logging
 - Heavy operations

Examples:

- Task assignment notification
- Daily summary emails

9. Deliverables

By the deadline, you must submit:

1. Fully functional API system
2. Postman collection (or similar tool) for testing APIs
3. Database schema (ERD or diagram)
4. GitHub repository with:
 - Clean code
 - Proper commits
 - README file
5. Deployed backend (optional but strongly recommended)

10. README Requirements

Your README must include:

- Project overview
- Architecture explanation
- Database design
- API documentation (endpoints, request/response)
- Setup instructions
- Example credentials

11. Evaluation Criteria

Your project will be evaluated based on:

11.1 Security Implementation

- Proper authentication and authorization
- Correct tenant isolation
- No data leakage between companies

11.2 Database Design

- Proper relationships
- Normalized structure
- Efficient queries

11.3 API Design

- Clean and RESTful endpoints
- Proper status codes
- Consistent request/response formats

11.4 Code Quality

- Clean, readable, and modular code
- Proper use of Django best practices

11.5 Scalability Thinking

- Can this system handle more tenants?
- Is the design extendable?

12. Important Instructions

- Do not hardcode tenant logic
- Always think in terms of scalability
- Test your APIs thoroughly
- Follow proper commit practices
- Focus on clean architecture, not just functionality